

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-13 (ebe6e5e+수정중) → 2026-08-21 · fe6c42d+수정중 | 1개 파일 · +9 / -2 줄 · 코드 4쪽 | 새 코드북 71쪽

바뀐 내용

- 파일명에 엔디안 토큰이 없을 때의 기본값을 환경변수 SIGNUS_ENDIAN 으로 정한다. 실신호 장비의 녹음기는 빅엔디안이라(2026-08-21 확정) 장비 셸에 export SIGNUS_ENDIAN=be 한 줄을 넣으면 파일 이름을 바꾸지 않아도 전부 바르게 읽힌다. .be/.le 토큰은 언제나 우선한다. 값이 le/be 가 아니면 조용히 넘어가지 않고 오류를 낸다.

□ `signus/sigio.py` +9 -2 2쪽 코드북 4쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/sigio.py +9 -2 · 코드북 4쪽 · 새로 쓸 줄 3-4, 83-89

```

1 1 """Sample file I/O. Filename carries read-necessities ONLY (fs, cplx|real, sample
2 2 type, endian, bitrev); ground truth lives in a '<filename>.json' sidecar the
3 + analyzer never reads. SigMF sidecars ('<stem>.sigmf-meta') are also understood.
4 + Endianness falls back to SIGNUS_ENDIAN when the filename carries no .be/.le token."""
5
6 import json
7 import os
8 import re
9 from dataclasses import dataclass
10
11 import numpy as np
12
13 # fs/rf 토막은 옛 밑줄 형식 전용이다. 점으로도 끊길 수 있어 경계에 둘 다 넣는다 -- 새 형식은
14 # 샘플레이트를 접두사 없이 cplx|real 바로 뒤에 놓으므로 이 정규식에 걸리지 않는다.
15 _FS_TOK = re.compile(r"(?:^|[_])fs(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[:[_]|$)", re.I)
16 _RF_TOK = re.compile(r"(?:^|[_])rf(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[:[_]|$)", re.I)
17
18 # token -> (numpy code, full-scale divisor, offset). u-types are offset binary
19 # (e.g. 80 / RTL-SDR u8: 0..255 with 128 = zero).
20 _DTYPES = {
21     "i8": ("i1", 128.0, 0.0), "u8": ("u1", 128.0, 128.0),
22     "i16": ("i2", 32768.0, 0.0), "u16": ("u2", 32768.0, 32768.0),
23     "f32": ("f4", 1.0, 0.0), "f64": ("f8", 1.0, 0.0),
24 }
25 # baudline-style aliases: <bits>t = two's complement, <bits>o = offset binary
26 _ALIAS = {"8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64"}
27 _TOK = {v: k for k, v in _ALIAS.items()} # 저장할 땐 baudline 표기로 되돌린다
28 _FMT = {"cplx": "iq", "real": "real", "iq": "iq"} # iq = 옛 밑줄 형식의 이름
29 _BITREV = np.array([int(f"{i:08b}"[::-1]), 2] for i in range(256)], dtype=np.uint8)
30
31 # SigMF core:datatype -> (dtype token, fmt, endian)
32 _SIGMF = {"cf32": ("f32", "iq"), "ci16": ("i16", "iq"), "ci8": ("i8", "iq"),
33     "cu8": ("u8", "iq"), "rf32": ("f32", "real"), "ri16": ("i16", "real")}
34
35 @dataclass
36 class Meta:
37     fs: float | None = None
38     fmt: str | None = None # 'iq' | 'real'
39     dtype: str = "i16" # key of _DTYPES
40     endian: str = "le" # 'le' | 'be' (floats included)
41     bitrev: bool = False # bit order reversed within each byte
42     rf_center: float | None = None # RF centre freq (Hz); None = unknown (report baseband)
43
44     def ok(self) -> bool:
45         return self.fs is not None and self.fmt in ("iq", "real") and self.dtype in _DTYPES
46
47     def parse_name(name: str) -> Meta:
48         """파일명이 읽기 필수 정보를 실어 나른다 (정답은 사이드카에만 있고 절대 읽지 않는다).
49
50         새 형식 <이름>.cplx|real.<샘플레이트 정수>.<16t>[.be][.bitrev].pcm
51         옛 형식 <이름>_fs<샘플레이트>_iq|real_<i16>[_be][_bitrev].iq
52
53         샘플레이트는 새 형식에서 접두사 없이 온다 -- 그래서 '숫자처럼 생긴 토막'이 아니라
54         cplx|real **바로 다음 자리**로 찾는다. 이름 자체가 숫자여도(20260801.cplx...) 안 헛갈린다.
55         확장자를 떼지 않고 통째로 쪼갬다: 새 형식은 마지막 토막(.pcm)이 포맷을 뜻하지 않고,
56         옛 형식은 확장자(.iq)가 이미 다른 토막과 같은 말을 하므로 어느 쪽도 손해가 없다."""
57         base = os.path.basename(name).lower()

```

```

59 60     toks = re.split(r"[._]", base)
60 61     # 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 **마지막** 후보. 세 오독을 막는다
61 62     # · 라벨의 real/cplx/iq(rec_iq_20260801.cplx...)가 fs/fmt 를 강탈 -- 마지막 후보 규칙이 막고
62 63     # · 점 긴 샘플레이트(cap.cplx.2.4e6... -> fs=2 Hz)와 라벨 숫자를 fs 로 발명(...iq_20260728.ra
63 64     # -- 숫자 뒤가 제원 토막(16t/be/.../pcm)이어야 한다는 관문이 막는다. 관문에 걸리면 fs 없음
64 65     # 크게 죽는다: 조용한 오답 대신 시끄러운 실패가 이 저장소의 원칙이다.
65 66     cands = [k for k, t in enumerate(toks) if t in _FMT]
66 67     hits = [k for k in cands if k + 2 < len(toks) and toks[k + 1].isdigit()
67 68             and (toks[k + 2] in _DTYPES or toks[k + 2] in _ALIAS
68 69             or toks[k + 2] in ("be", "bitrev", "pcm") or toks[k + 2].startswith("rf"))]
69 70     i = hits[-1] if hits else (cands[0] if cands else None)
70 71     m = Meta()
71 72     if i is not None:                                # 'is not None' 이어야 한다 -- 0 번 토막(cplx...)도 유효
72 73         # 하다
73 74         m.fmt = _FMT[toks[i]]
74 75         if i in hits:
75 76             m.fs = float(toks[i + 1])
76 77         if m.fs is None and (mt := _FS_TOK.search(base)):
77 78             m.fs = float(mt.group(1))
78 79         if rt := _RF_TOK.search(base):
79 80             m.rf_center = float(rt.group(1))
80 81         tail = toks[i + 1:] if i is not None else toks    # 제원은 포맷 토막 **뒤**에만 산다 --
81 82         m.dtype = next(((_ALIAS.get(t, t) for t in tail    # 라벨의 u8/f32/be 가 제원을 오염 못 하게
                        if t in _DTYPES or t in _ALIAS), "i16"))
83 +     # 엔디안: 파일명 토큰(.be/.le)이 최우선, 없으면 SIGNUS_ENDIAN, 그것도 없으면 le.
84 +     # 실장비 녹음기는 BE 라 장비에서는 export SIGNUS_ENDIAN=be 한 줄로 전 파일이 읽힌다
85 +     # (2026-08-21: BE 캡처를 LE 로 읽어 균등분포 잡음으로 보이던 사고 뒤에 넣었다).
86 +     env = os.environ.get("SIGNUS_ENDIAN", "le").lower()
87 +     if env not in ("le", "be"):
88 +         raise ValueError(f"SIGNUS_ENDIAN 은 le 또는 be 여야 합니다: {env!r}")
89 +     m.endian = "be" if "be" in tail else ("le" if "le" in tail else env)
83 90     m.bitrev = "bitrev" in tail
84 91     return m
85 92
86 93
87 94 def parse_sigmf(path: str) -> Meta | None:
88 95     """Read `<stem>.sigmf-meta` (core:sample_rate + core:datatype) if present."""
89 96     try:
90 97         with open(os.path.splitext(path)[0] + ".sigmf-meta") as fh:
91 98             doc = json.load(fh)
92 99             g = doc["global"]
93 100             code = g["core:datatype"].replace("_le", "").replace("_be", "")
94 101             dtype, fmt = _SIGMF[code]
95 102             endian = "be" if g["core:datatype"].endswith("_be") else "le"
96 103             fs = float(g["core:sample_rate"])
97 104         except (OSError, KeyError, ValueError, TypeError, AttributeError):
98 105             return None # MANDATORY fields malformed: fall back to filename tokens
99 106     try:
100 107         # rf centre is OPTIONAL: a malformed value must cost only this field, never the whole
101 108         # sidecar -- discarding valid fs/fmt/dtype here silently re-read the file with filename
102 109         # tokens, which can lie about fs (a quiet garbage decode).
103 110         caps = doc.get("captures") or []
104 111         rf = caps[0].get("core:frequency") if caps else None
105 112         rf = None if rf is None else float(rf)
106 113     except (KeyError, ValueError, TypeError, AttributeError, IndexError):
107 114         rf = None

```

```

108 115         return Meta(fs, fmt, dtype, endian, rf_center=rf)
109 116
110 117
111 118 def make_name(label: str, m: Meta) -> str:
112 119     """저장은 새 형식만: <이름>.cplx|real.<샘플레이트>.<16t>[.be][.bitrev].pcm"""
113 120     extra = (".be" if m.endian == "be" else "") + (".bitrev" if m.bitrev else "")
114 121     kind = "real" if m.fmt == "real" else "cplx"
115 122     return f"{label}.{kind}.{m.fs:.0f}._{TOK.get(m.dtype, m.dtype)}{extra}.pcm"
116 123
117 124
118 125 def decode(raw: bytes, meta: Meta) -> np.ndarray:
119 126     """Raw bytes -> samples: complex128 for 'iq', float64 for 'real'.
120 127     Tolerates truncated files (drops trailing partial samples)."""
121 128     code, scale, off = _DTYPES[meta.dtype]
122 129     if meta.bitrev:
123 130         raw = _BITREV[np.frombuffer(raw, dtype=np.uint8)].tobytes()
124 131     step = np.dtype(code).itemsize
125 132     dt = ("<" if meta.endian == "le" else ">") + code
126 133     x = np.frombuffer(raw[: len(raw) - len(raw) % step], dtype=dt).astype(np.float64)
127 134     x = (x - off) / scale
128 135     if meta.fmt == "iq":
129 136         x = x[: x.size & ~1] # drop dangling half-sample
130 137         return x[0::2] + 1j * x[1::2]
131 138     return x
132 139
133 140
134 141 def read(path: str, meta: Meta | None = None) -> tuple[np.ndarray, Meta]:
135 142     meta = meta or parse_sigmf(path) or parse_name(path)
136 143     if not meta.ok():
137 144         raise ValueError(f"need fs + fmt (filename tokens, SigMF, or --fs/--fmt): {path}")
138 145     with open(path, "rb") as fh:
139 146         x = decode(fh.read(), meta)
140 147     if x.size < 256:
141 148         raise ValueError(f"신호가 너무 짧습니다 ({x.size} samples): {path}")
142 149     return x, meta
143 150
144 151
145 152 def write(path: str, x: np.ndarray, meta: Meta) -> None:
146 153     """Quantize (ints: scaled to 99.9th-percentile full scale) and write interleaved."""
147 154     code, scale, off = _DTYPES[meta.dtype]
148 155     flat = np.column_stack([x.real, x.imag]).ravel() if np.iscomplexobj(x) else x.real
149 156     if scale != 1.0: # integer types
150 157         peak = np.percentile(np.abs(flat), 99.9) or 1.0
151 158         flat = np.round(flat * (scale - 1) / peak) + off
152 159         info = np.iinfo(code)
153 160         flat = np.clip(flat, info.min, info.max)
154 161     out = flat.astype(("<" if meta.endian == "le" else ">") + code)
155 162     if meta.bitrev:
156 163         out = np.frombuffer(_BITREV[np.frombuffer(out.tobytes(), np.uint8)].tobytes(),
157 164                             dtype=out.dtype)
158 165     out.tofile(path)
159 166
160 167
161 168 def sidecar_write(path: str, meta: Meta, truth: dict, gen: dict) -> None:
162 169     doc = {"fs": meta.fs, "fmt": meta.fmt, "dtype": meta.dtype,
163 170           "endian": meta.endian, "bitrev": meta.bitrev, "truth": truth, "gen": gen}
164 171     with open(path + ".json", "w") as fh:
165 172         json.dump(doc, fh, indent=1)
166 173
167 174

```

```
168 175 def sidecar_read(path: str) -> dict | None:
169 176     try:
170 177         with open(path + ".json") as fh:
171 178             return json.load(fh)
172 179     except (OSError, ValueError):
173 180         return None
```